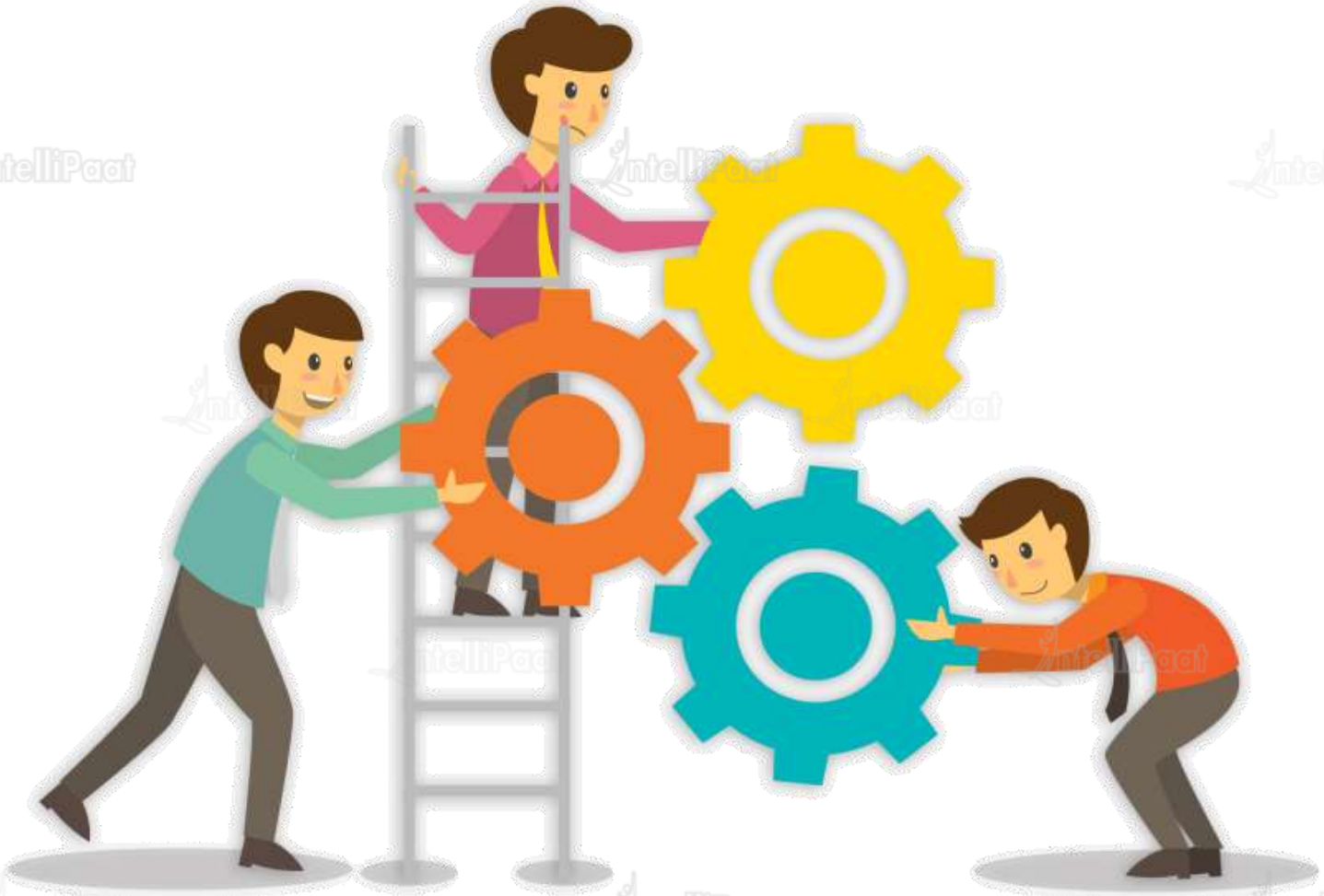
Python Training

Python Modules



Copyright Intellipaat. All rights reserved.

Agenda

01 Creating a Modules

02 Importing Modules

03 More on Modules and Importing

04 Executing Modules as Script

05 The Module Search Path

06 Complied Python Files

07 Standard Modules

08 The dir() function

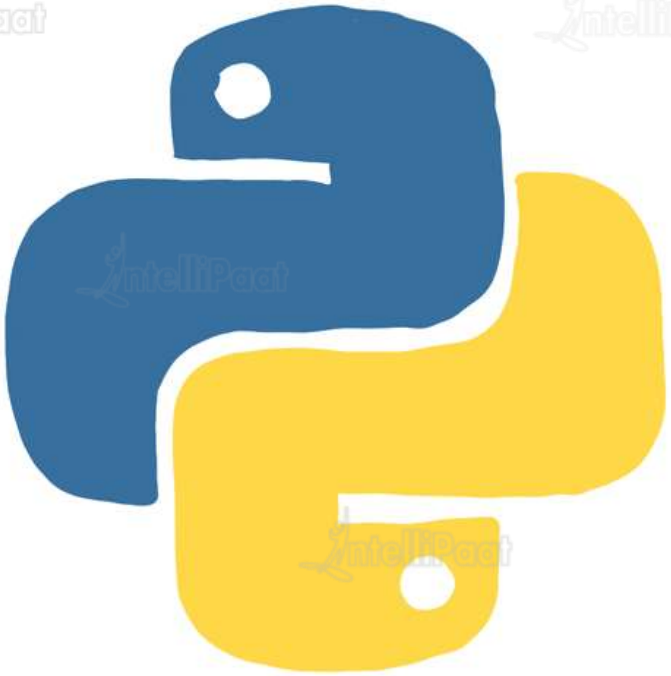
Copyright Intellipaat. All rights reserved.

Creating a Module

Copyright Intellipaat. All rights reserved.

Python Module

- Consider a module to be the same as a code library.
- A file containing a set of functions you want to include in your application.
- A module allows you to logically organize your Python code.
- Grouping related code into a module makes the code easier to understand and use.
- A module is a Python object with arbitrarily named attributes that you can bind.
- Simply, a module is a file consisting of Python code.



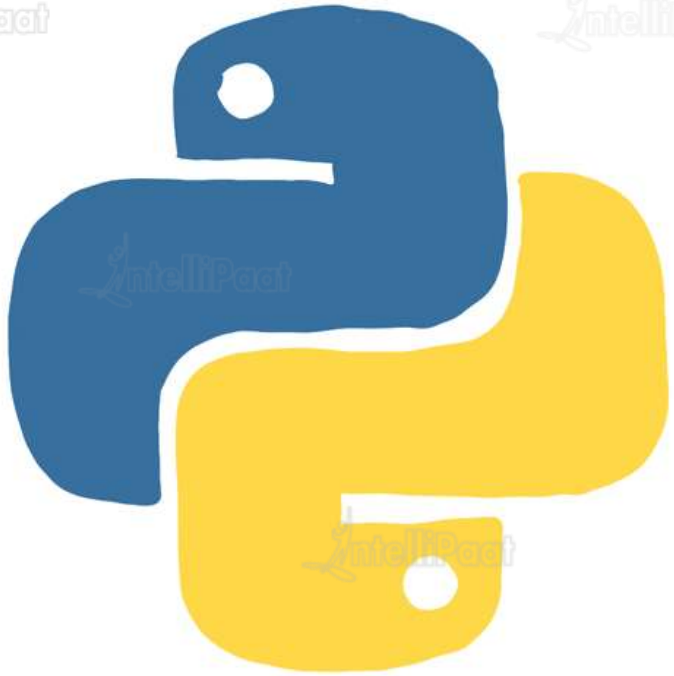
Modules

Copyright Intellipaat. All rights reserved.

Python Module

- A module is a file containing Python definitions and statements.
- A module can define functions, classes and variables. A module can also include runnable code.
- Grouping related code into a module makes the code easier to understand and use.
- Example:-

```
# A simple module, calc.py
def add(x, y):
return (x+y)
def subtract(x, y):
return (x-y)
```



Modules

Copyright Intellipaat. All rights reserved.

How to import Modules in Python?

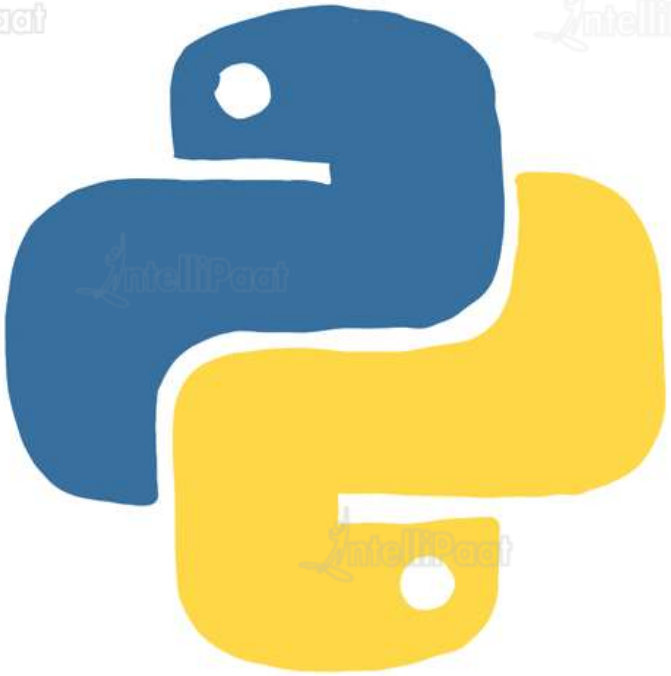
- We can import the definitions inside a module to another module or the interactive interpreter in Python.

- We use the import keyword to do this. To import our previously defined module example we type the following in the Python prompt.
  >>> import example

- This does not enter the names of the functions defined in example directly in the current symbol table.

- It only enters the module name example there.

- Using the module name we can access the function using the dot . operator. For example:
  >>> example.add(4,5.5)
  9.5
- Standard modules can be imported the same way as we import our user-defined modules.



Modules

Copyright Intellipaat. All rights reserved.

The Import statement

- You can use any Python source file as a module by executing an import statement in some other Python source file.

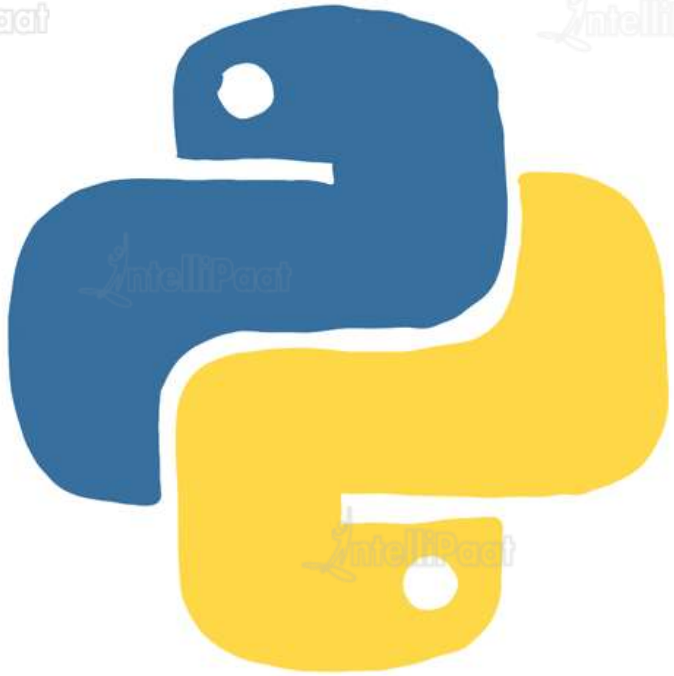
- The import has the following syntax –
- import module1[, module2[,... moduleN]

- When the interpreter encounters an import statement, it imports the module if the module is present in the search path.

- A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module support.

- You need to put the following command at the top of the script –

```
#!/usr/bin/python
# Import module support
import support
# Now you can call defined function that module as follows
support.print_func("Zara")
```

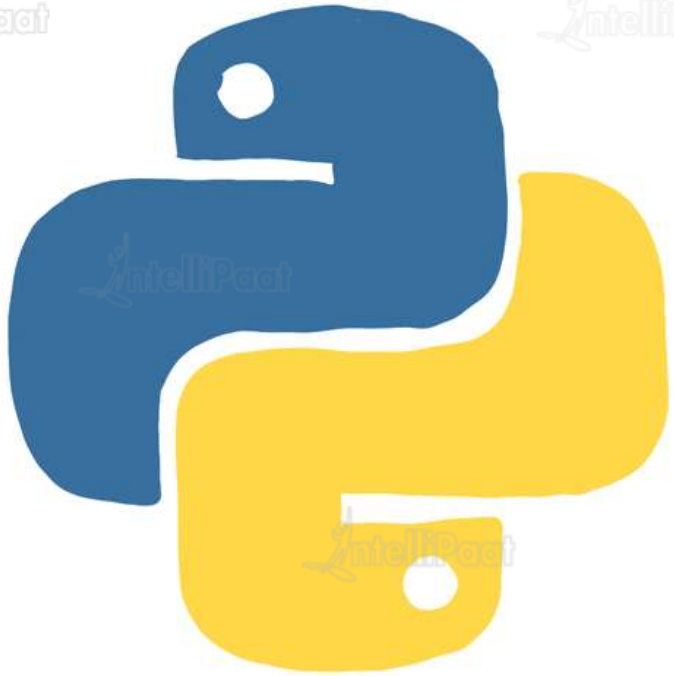


Modules

Copyright Intellipaat. All rights reserved.

The from… Import statement

- Python from statement lets you import specific attributes from a module into the current namespace.
- The from...import has the following syntax −
  from modname import name1[, name2[, ... nameN]]
- For example, to import the function fibonacci from the module fib, use the following statement −
  from fib import fibonacci
- This statement does not import the entire module fib into the current namespace; it just introduces the item fibonacci from the module fib into the global symbol table of the importing module.
- Python from statement lets you import specific attributes from a module.
- This provides an easy way to import all the items from a module into the current namespace; however, this statement should be used sparingly.

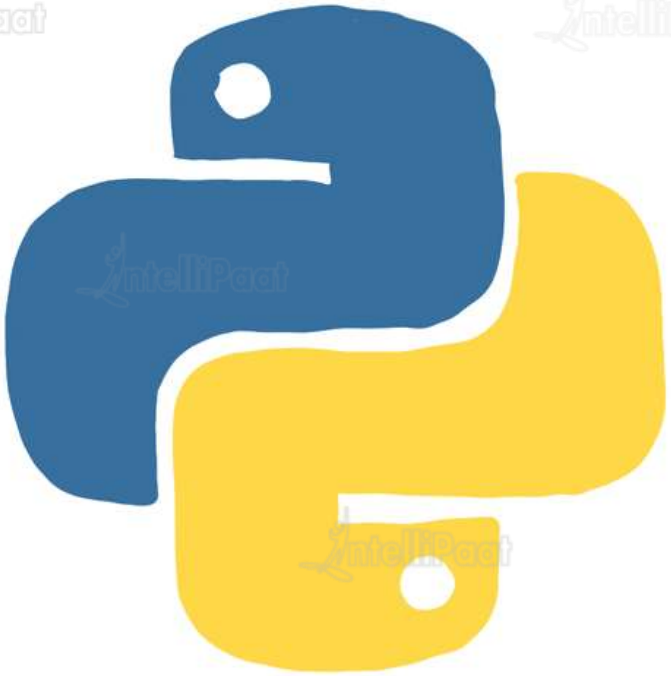


Modules

Copyright Intellipaat. All rights reserved.

Locating Module

- When you import a module, the Python interpreter searches for the module in the following sequences −
- The current directory.
- If the module isn't found, Python then searches each directory in the shell variable PYTHONPATH.
- If all else fails, Python checks the default path. On UNIX, this default path is normally /usr/local/lib/python/.
- The module search path is stored in the system module sys as the sys.pathvariable.
- The sys.path variable contains the current directory, PYTHONPATH, and the installation-dependent default.

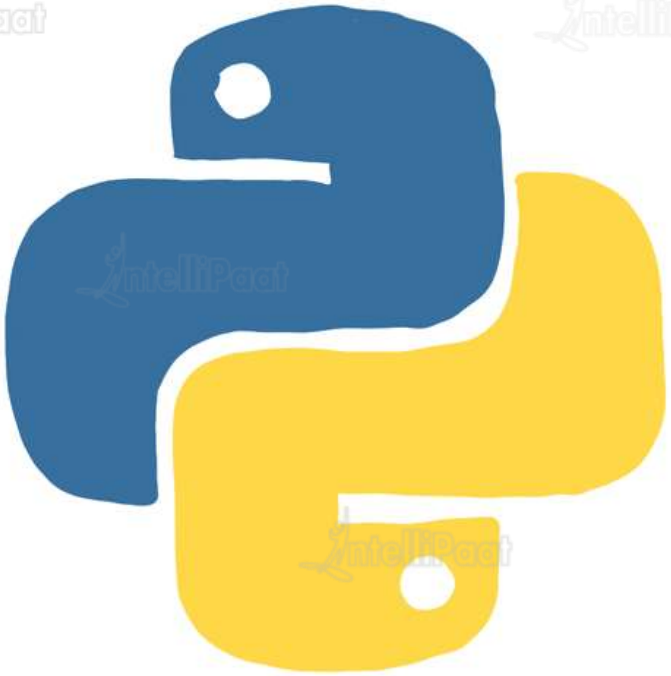


Modules

Copyright Intellipaat. All rights reserved.

Namespace and Scoping

- Variables are names (identifiers) that map to objects.
- A namespace is a dictionary of variable names (keys) and their corresponding objects (values).
- A Python statement can access variables in a local namespace and in the global namespace.
- If a local and a global variable have the same name, the local variable shadows the global variable.
- Each function has its own local namespace.
- Class methods follow the same scoping rule as ordinary functions.
- Python makes educated guesses on whether variables are local or global.
- It assumes that any variable assigned a value in a function is local.

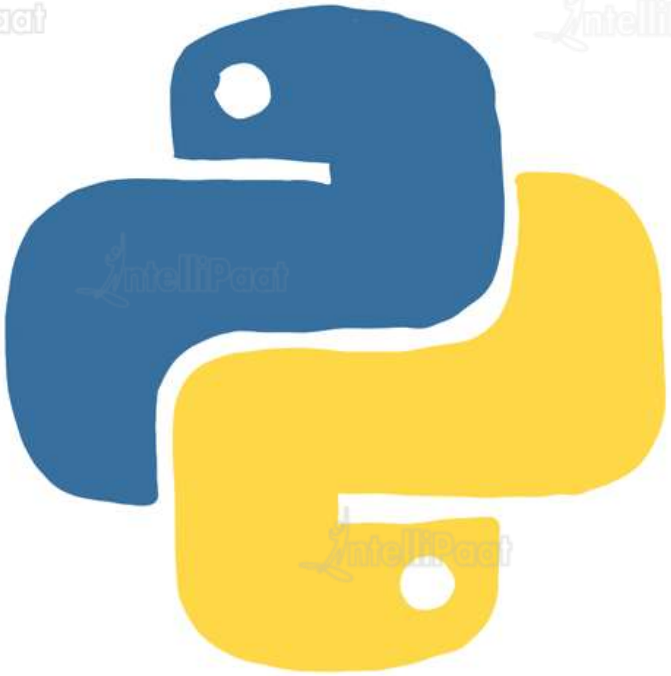


Modules

Copyright Intellipaat. All rights reserved.

Namespace and Scoping

- Therefore, in order to assign a value to a global variable within a function, you must first use the global statement.
- The statement global VarName tells Python that VarName is a global variable.
- Python stops searching the local namespace for the variable.
- For example, we define a variable Money in the global namespace.
- Within the function Money, we assign Money a value, therefore Python assumes Money As a local variable.
- However, we assessed the value of the local variable Money Before setting it, so an UnboundLocalError is the result.
- Uncommenting the global statement fixes the problem.



Modules

Copyright Intellipaat. All rights reserved.

Python module search path

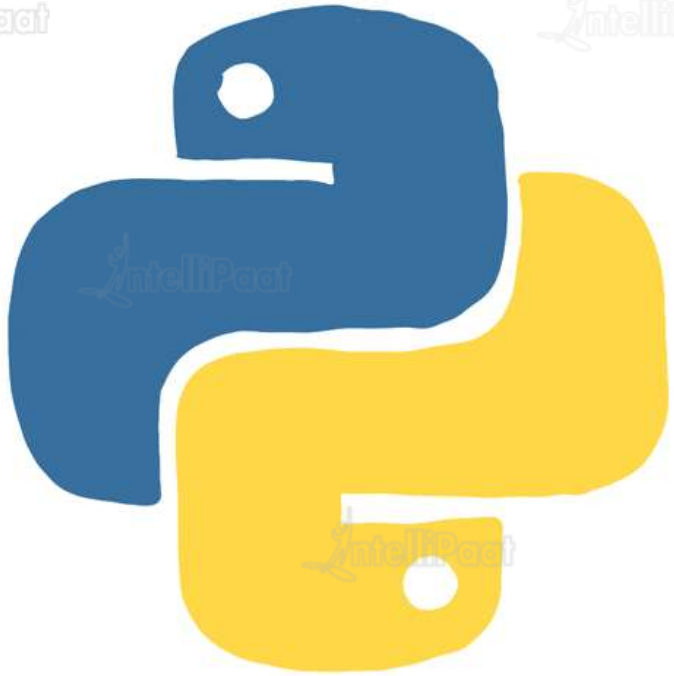
- While importing a module, Python looks at several places.
- Interpreter first looks for a built-in module then (if not found) into a list of directories defined in sys.path.
- The search is in this order.
- The current directory.
- PYTHONPATH (an environment variable with a list of directory).
- The installation-dependent default directory.
- We can add modify this list to add our own path.

```
>>> import sys
>>> sys.path
['',
'C:\\Python33\\Lib\\idlelib',
'C:\\Windows\\system32\\python33.zip',
'C:\\Python33\\DLLs',
'C:\\Python33\\lib',
'C:\\Python33',
'C:\\Python33\\lib\\site-packages']
```

Copyright Intellipaat. All rights reserved.

The global() and local() fucntion

- The globals() and locals() functions can be used to return the names in the global and local namespaces depending on the location from where they are called.
- If locals() is called from within a function, it will return all the names that can be accessed locally from that function.
- If globals() is called from within a function, it will return all the names that can be accessed globally from that function.
- The return type of both these functions is dictionary. Therefore, names can be extracted using the keys() function.
- The globals() function returns a dictionary containing the variables defined in theglobal namespace.
- When globals() is called from a function or method, it returns the dictionary representing the global namespace of the module.

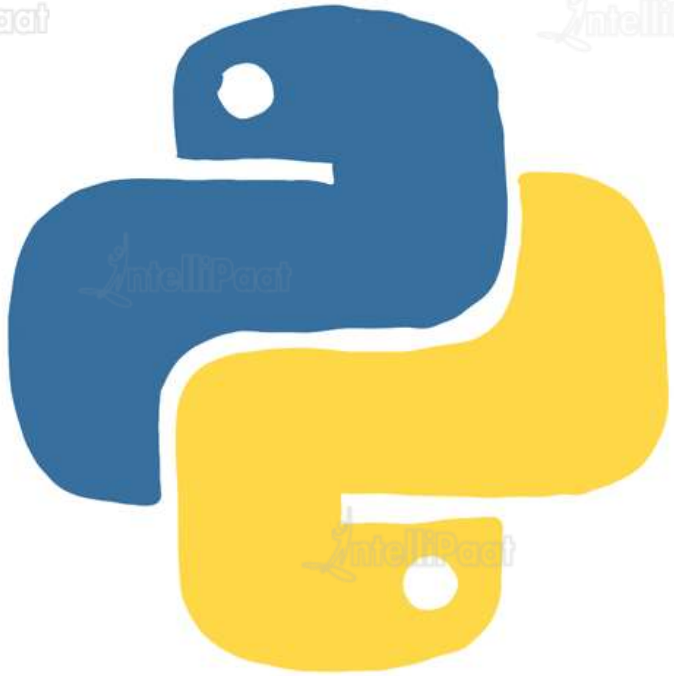


Modules

Copyright Intellipaat. All rights reserved.

The reload functions

- The Python interpreter imports a module only once during a session. This makes things more efficient.
- When the module is imported into a script, the code in the top-level portion of a module is executed only once.
- Therefore, if you want to re execute the top-level code in a module, you can use the reload() function.
- The reload() function imports a previously imported module again. The syntax of the reload() function is this –
- reload(module_name)
- Here, module_name is the name of the module you want to reload and not the string containing the module name.
- For example, to reload hello module, do the following –
- reload(hello)



Modules

Copyright Intellipaat. All rights reserved.

India: +91-7847955955

US: 1-800-216-8930 (TOLL FREE)

support@intellipaat.com

24/7 Chat with Our Course Advisor

Copyright Intellipaat. All rights reserved.